IMPLEMENTAREA CONCURENTEI IN LIMBAJE DE PROGRAMARE

Ioana Leustean

INTRODUCERE IN ERLANG

http://www.erlang.org/

➢Bibliografie

Joe Armstrong, Programming Erlang, Second Edition 2013

Fred Hébert, Learn You Some Erlang For Great Good, 2013 Varianta online

➢ Tratarea erorilor

Error handling in concurrent Erlang programs is based on the idea of *remote detection and handling of errors*. Instead of handling an error in the process where the error occurs, we let the process die and correct the error in some other process."
Joe Armstrong, Programming Erlang, Second Edition 2013

➢ OTP

OTP stands for Open Telecom Platform, although it's not that much about telecom anymore (it's more about software that has the property of telecom applications, but yeah.) If half of Erlang's greatness comes from its concurrency and distribution and the other half comes from its error handling capabilities, then the OTP framework is the third half of it.
http://learnyousomeerlang.com/what-is-otp#its-the-open-telecom-platform

```
-module(myserver).
-export([start/2, client/2]).

start(Name, Mod) ->  register(Name, spawn(fun() -> loop(Name, Mod) end)).

client(Name, Request) ->
   Name ! {self(), Request},
   receive
      {Name, Response} -> Response
   end.
loop(Name, Mod) ->
   receive
   {From, Request} ->  Response = Mod:handle(Request),
                       From ! {Name, Response},
                        loop(Name, Mod)
   end.
```

comportamentul de baza al clientului

comportamentul de baza al serverului, parametrizat cu un modul **Mod**

Modulul "callback" nu contine nimic legat de concurenta!

```
-module(callback_mod).
-export([client1/1, client2/1, handle/1]).
-import(myserver, [client/2]).
```

interfata clientului

```
%% client
client1(N) -> client(myserver, {add, N}).
client2(Name) -> client(myserver, {hello, Name}).
```

functiile callback care sunt apelate din modulul serverului

```
%% callback
handle({add, N}) -> {ok, N+1};
handle({hello, Name})      -> {ok,"hello "++ Name}.
```

```
-module(callback_mod).
-export([client1/1, client2/1, handle/1]).
-import(myserver, [client/2]).
```

Modulul "callback" nu contine nimic legat de concurenta!

interfata clientului

```
%% client
client1(N) -> client(myserver, {add, N}).
client2(Name) -> client(myserver, {hello,
```

functiile callback care sunt apelate din modulul serverului

```
%% callback
handle({add, N}) -> {ok, N+1};
handle({hello, Name})      -> {ok,"hello "++ Name}.
```

```
1> c(myserver).
{ok,myserver}
2> c(callback_mod).
{ok,callback_mod}
3> myserver:start(myserver,callback_mod).
true
4> callback_mod:client1(6).
{ok,7}
5> callback_mod:client2("Ioana").
{ok,"hello Ioana"}
```

- Separam codul pur functional de partea comportamentala (behaviour)

```
1> c(myserver).
{ok,myserver}
2> c(callback_mod).
{ok,callback_mod}
3> myserver:start(myserver,callback_mod).
true
4> callback_mod:client1(6).
{ok,7}
5> callback_mod:client2("Ioana").
{ok,"hello Ioana"}
```

Modulul server descrie comportamentul de baza, continue codul concurent si este parametrizat de modulul callback

Modulul "callback" nu contine nimic legat de concurenta!

```
-module(callback_mod).
-export([client1/1, client2/1, handle/1]).
-import(myserver, [client/2]).


%% client
client1(N) -> client(myserver, {add, N}).
client2(Name) -> client(myserver, {hello, Name}).


%% callback
handle({add, N}) -> {ok, N+1};
handle({hello, Name})      -> {ok,"hello "++ Name}.
```

```
module(myserver).
-export([start/2, client/2]).

start(Name, Mod) ->  register(Name, spawn(fun() -> loop(Name,
Mod) end)).

client(Name, Request) ->
   Name ! {self(), Request},
   receive
      {Name, Response} -> Response
   end.
loop(Name, Mod) ->
   receive
   {From, Request} ->  Response = Mod:handle(Request),
                       From ! {Name, Response},
                        loop(Name, Mod)
   end.
```

- Server cu tratarea erorilor

```
-module(myservertc).
-export([start/2, client/2]).

start(Name, Mod) ->
   register(Name, spawn(fun() -> loop(Name, Mod) end)).

client(Name, Request) ->
   Name ! {self(), Request},
   receive
      {Name, Response} -> Response;
      {Name, myError} -> exit(myError)
   end.
```

```
loop(Name, Mod) ->
   receive
   {From, Request} ->
         try  Mod:handle(Request) of
          Response -> From ! {Name, Response},
                  loop(Name, Mod)
         catch
           _:Reason ->  io:format("Server  ~p exception ~p~n",
                        [Name, Reason]),
           From ! {Name, myError},
           loop(Name, Mod)
         end
```

```
-module(callback_mod_er).
-export([client1/1, client2/1, client3/2, handle/1]).
-import(myservertc, [client/2]).


%% client
client1(N) -> client(myserver, {add, N}).
client2(Name) -> client(myserver, {hello, Name}).
client3(N,M) -> client(myserver,{division,N,M}).


%% callback
handle({add, N}) -> {ok, N+1};
handle({hello, Name})      -> {ok,"hello "++ Name};
handle({division, N,M})      -> {ok,N / M}.
```

```
-module(callback_mod_er).
-export([client1/1, client2/1, client3/2
-import(myservertc, [client/2]).


%% client
client1(N) -> client(myserver, {add, N}
client2(Name) -> client(myserver, {hel
client3(N,M) -> client(myserver,{divisi


%% callback
handle({add, N}) -> {ok, N+1};
handle({hello, Name})      -> {ok,"hello
handle({division, N,M})      -> {ok,N / M}.
```

```
5> c(callback_mod_er).
{ok,callback_mod_er}
6> myservertc:start(myserver,callback_mod_er).
true
7> callback_mod_er:client1(4).
{ok,5}
8> callback_mod_er:client3(4,2).
{ok,2.0}
9> callback_mod_er:client3(4,0).
Server  myserver exception badarith
myError
10> callback_mod_er:client2("Ioana").
{ok,"hello Ioana"}
```

- Server cu schimbarea comportamentului

```
-module(myserver2).
-export([start/2, client/2,change_behaviour/2]).

start(Name, Mod) ->
   register(Name, spawn(fun() -> loop(Name, Mod) end))

client(Name, Request) ->
   Name ! {self(), Request}
   receive
      {Name, Response} ->
   end.
```

```
change_behaviour(Name, Mod) -> client(Name, {change, Mod}).

loop(Name, Mod) ->
   receive
   {From, {change, NewCallBack}} ->   From ! {Name, i_will_change},
                                      loop(Name, NewCallBack);
   {From, Request} ->  Response = Mod:handle(Request),
                       From ! {Name, Response},
                       loop(Name, Mod)
   end.
```

- Server cu schimbarea comportamentului: mai multe module callback

```
-module(callback_mod2).
-export([client1/1, client2/1, handle/1]).
-import(myserver2, [client/2]).


%% client
client1(N) -> client(myserver, {add, N}).
client2(Name) -> client(myserver, {hello, Name}).


%% callback
handle({add, N}) -> {ok, N+1};
handle({hello, Name})      -> {ok,"hello "++ Name}.
```

```
-module(callback_new).
-export([client1/1, client2/1, handle/1]).
-import(myserver2, [client/2]).


%% client
client1(N) -> client(myserver, {multiply, N}).
client2(Name) -> client(myserver, {hate, Name}).


%% callback
handle({multiply, N}) -> {ok, N*N};
handle({hate, Name})      -> {ok,"I hate you,  "++ Name}.
```

- Schimbarea comportamentului serverului in mod dinamic

```
3> c(myserver2).
{ok,myserver2}
4> c(callback_mod2).
{ok,callback_mod2}
5> c(callback_new).
{ok,callback_new}
6> myserver2:start(myserver, callback_mod2).
true
7> callback_mod2:client1(5).
{ok,6}
8> callback_mod2:client2("Ana").
{ok,"hello Ana"}
9> myserver2:change_behaviour(myserver, callback_new).
i_will_change
10> callback_new:client2("Ana").
{ok,"I hate you,  Ana"}
11> callback_new:client1(5).
{ok,25}
```

"This is what OTP really is all about: taking all the generic components, extracting them in libraries, making sure they work well and then reusing that code when possible.

Adding servers adds complexity in terms of code, but also in terms of testing, maintenance and understanding. Each implementation might be different, programmed in different styles by different people, and so on. However, if all these servers share the same common my_server abstraction, you substantially reduce that complexity. You understand the basic concept of the module instantly ("oh, it's a server!"), there's a single generic implementation of it to test, document, etc. The rest of the effort can be put on each specific implementation of it.

Another interesting thing about what we did when separating the generic from the specific is that we instantly made it much easier to test our individual modules.

A much more 'hidden' advantage of using common abstractions in that way is that if everyone uses the exact same backend for their processes, when someone optimizes that single backend to make it a little bit faster, every process using it out there will run a little bit faster too. For this principle to work in practice, it's usually necessary to have a whole lot of people using the same abstractions and putting effort on them. Luckily for the Erlang community, that's what happens with the OTP framework."

https://learnyousomeerlang.com/what-is-otp#its-the-open-telecom-platform